

GUI-библиотека KVB_LCD для STM32F407 и ATmega128 без фрейм-буфера

Архитектура вопреки ограничениям

Когда речь заходит о создании графического интерфейса на микроконтроллерах, разработчики обычно упираются в жесткий выбор: либо использовать «тяжелые» библиотеки вроде **LVGL**, требующие мегабайты **RAM** под **фрейм-буфер** и мощный **Cortex-M**, либо соглашаться на примитивную графику для **8-битных** чипов.

Проект **KVB_LCD** доказывает, что при правильном знании геометрии и архитектуры дисплеев можно получить **тяжелые визуальные эффекты на лету**, используя исключительно **целочисленную математику** и **0 байт фрейм-буфера**. Графические объекты функционируют абсолютно идентично как на **32-битном STM32F407**, так и на народном **8-битном ATmega128**.

Главное доказательство: Скорость отображения

Вместо тысяч слов — просто посмотрите, как микроконтроллер без **фрейм-буфера** перерисовывает сложнейшие сцены:

- Смотреть демонстрацию работы на RUTUBE:

<https://rutube.ru/video/80140af8b045d20b939240ea3dab1d8b/>

- Смотреть демонстрацию работы на VK:

https://vkvideo.ru/video-241941150_456239018

- Смотреть демонстрацию работы на YouTube:

<https://youtu.be/XuP7KXyDUP8>

- Полный код проекта и описания можно скачать:

<https://github.com/Konstantin-kvb/KVB-Microcontroller-GUI-Library/>

На видео есть отображение **12-и трех поверхностных объектов**. Каждый из них состоит из **3-х независимых круглых/округлых градиентных поверхностей**. В сумме это **36 независимых градиентных поверхностей**, поверх которых накладываются **12 градиентных символов с честной тенью**, для большего эффекта света и тени центральная поверхность имеет смещение относительно наружной. И всё это отображается прямо в GRAM дисплея без **фрейм-буфера**.



Лицензия

Данная графическая библиотека распространяется под лицензией **Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)**.

Вы можете свободно использовать, изменять и распространять этот код в личных, образовательных и некоммерческих целях при условии указания авторства. Использование данной библиотеки в любых коммерческих проектах строго **запрещено**. По вопросам приобретения коммерческой лицензии обращайтесь по адресу [email kboruzdov@mail.ru tel +7 965-363-45-29].

Защита интеллектуальной собственности и проприетарные алгоритмы

Математические решения и алгоритмы, используемые в графическом ядре библиотеки, являются уникальной интеллектуальной собственностью автора. В частности, авторский алгоритм построения и заполнения градиентной окружности трех поверхностного объекта «**на лету**» с использованием исключительно целочисленной математики (без привлечения **фрейм-буфера**) является закрытым **ноу-хау** и потребовал огромных инженерных усилий. Это коммерчески ценная технология. Я не готов безвозмездно отдавать свои уникальные математические решения крупным компаниям.

В целях защиты авторских прав и предотвращения несанкционированного копирования уникальных низкоуровневых решений, графическое ядро библиотеки поставляется исключительно в виде предварительно скомпилированных закрытых бинарных модулей (файлы статических библиотек `.a`` и `.lib``).

Передача исходного кода математического ядра библиотеки (алгоритмов скоростной отрисовки линий, расчета градиентов **GRAM** и трех поверхностных объектов) не осуществляется ни при каких условиях, включая приобретение коммерческой лицензии. При этом разработчику предоставляется полный доступ к исходному коду интерфейсной **GUI**-оболочки (`KVB_LCD.h``). Она максимально подробно прокомментирована на русском языке, содержит описания всех функций, параметров, эффектов шрифтов и графических объектов.

Весь демонстрационный код снабжен развернутыми комментариями на русском языке для быстрого старта.

Кроссплатформенность

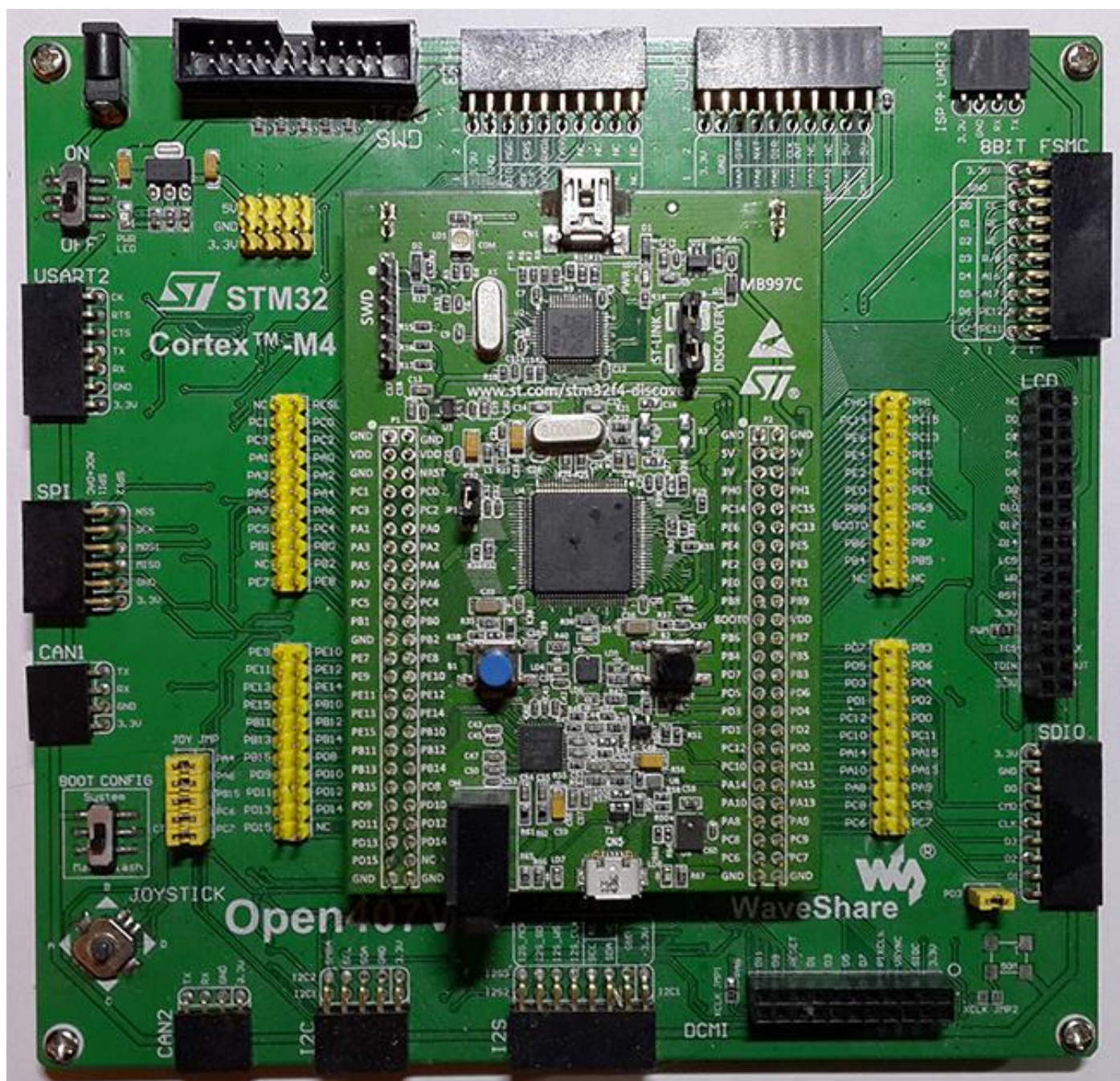
Библиотека написана с нуля на чистом языке **C** в **GCC** и **AC6** используя только целочисленную математику и не используя сторонние коды.

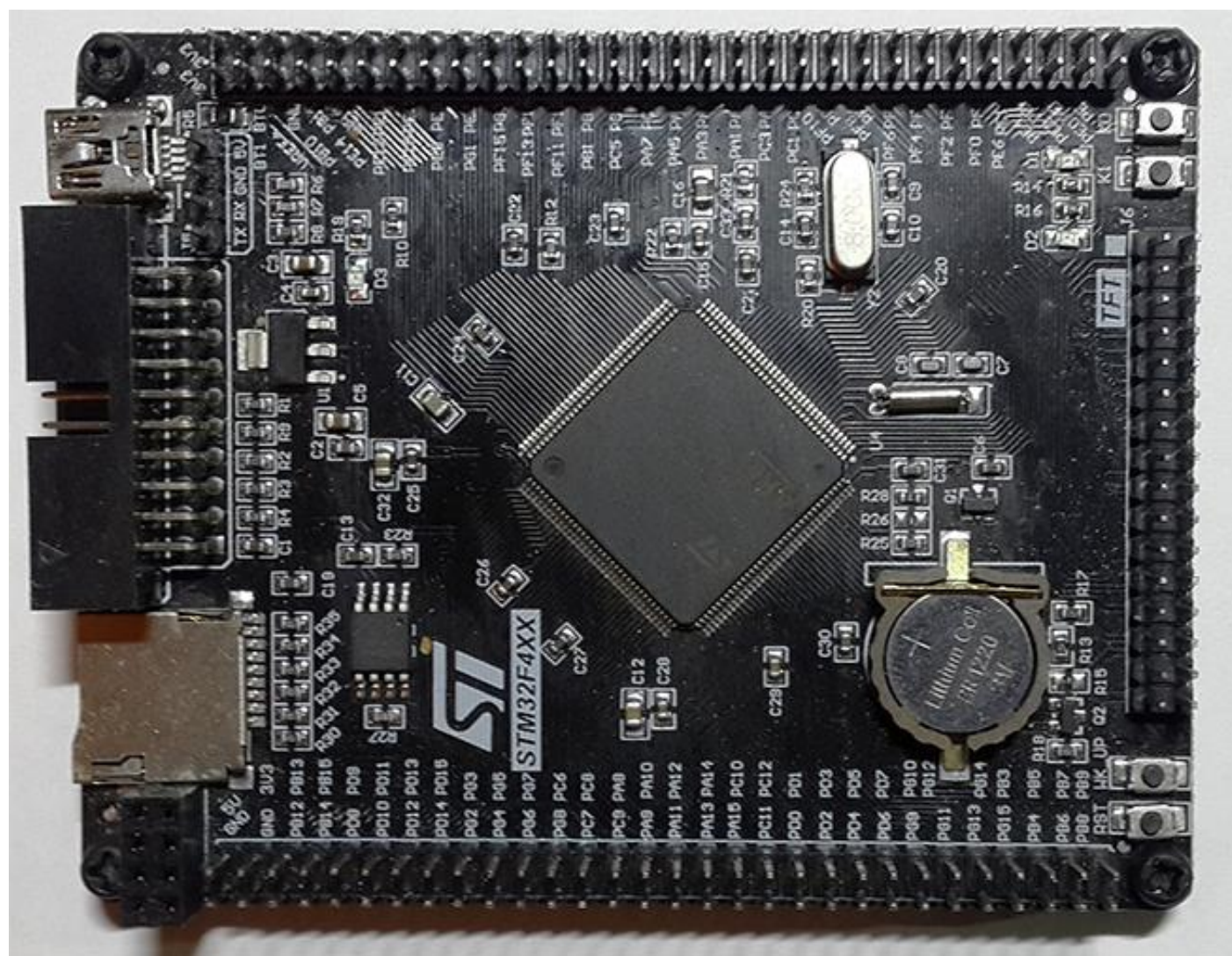
На данный момент библиотекой поддерживаются два микроконтроллера **ATmega128** и **STM32F407** на уровне графического интерфейса пользователя **GUI** (`KVB_LCD.h``). Графические объекты, написанные на основе **GUI** библиотеки, будут работать как на **8** ми битном микроконтроллере **ATmega128** так и на **32** х битном **STM32F407**.

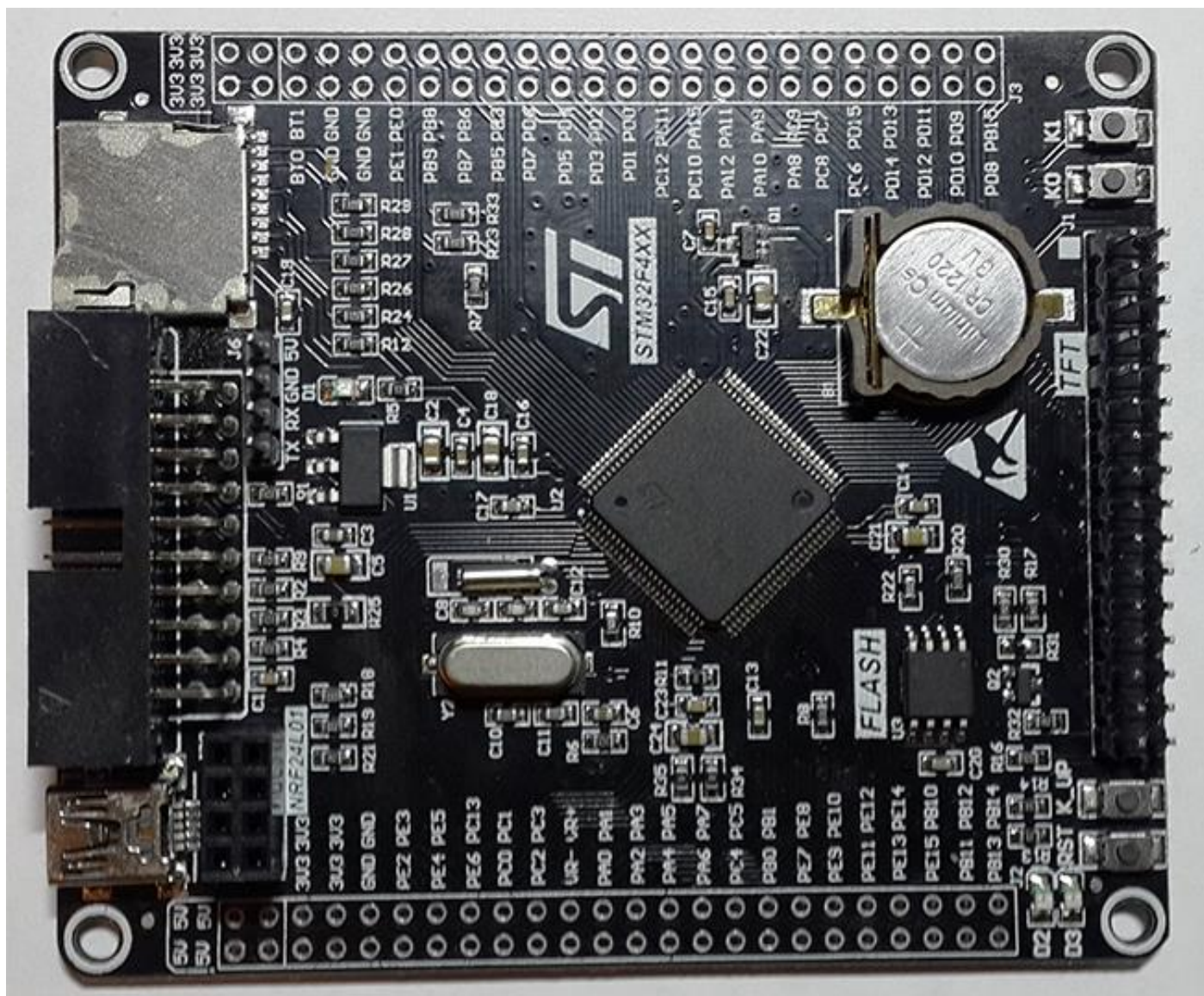
Для разработки программного кода библиотекой поддерживаются три среды разработки **IDE**; **Atmel Studio 7.0**, **STM32CubeIDE v2** и **Keil MDK v5.43**. Для просмотра

тестового презентационного кода под **STM32** поддерживаются три платы разработки с разъемом **FSMC**;

- **Open407V-D stm32f4-discovery** микроконтроллер **STM32F407VGT6**.
- **ST STM32F4XX Black v3.0 1606** микроконтроллер **STM32F407ZGT6**.
- **ST STM32F4XX Black v2.0 1509** микроконтроллер **STM32F407VET6**.

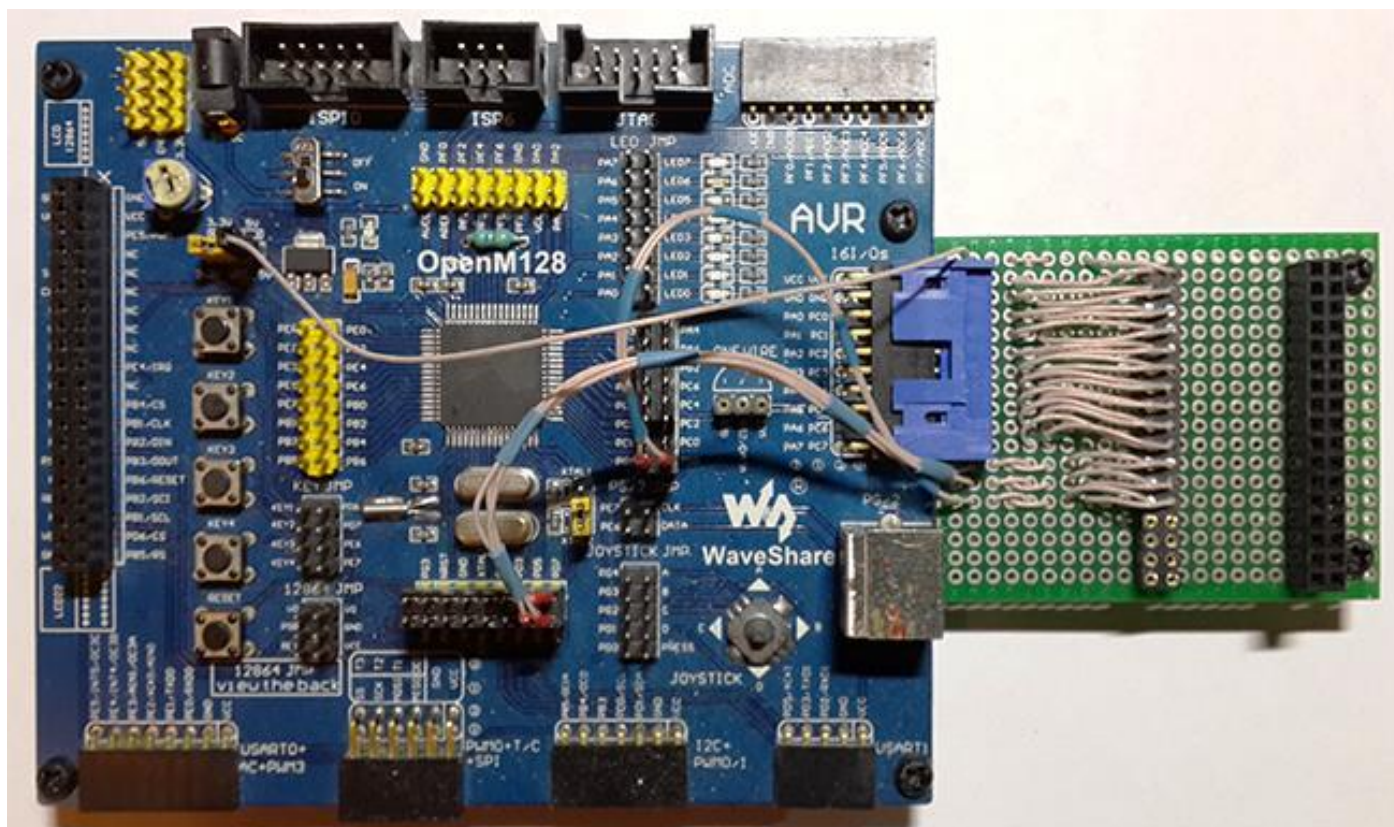






Для просмотра тестового презентационного кода под **ATmega128** используются порты ввода / вывода;

- **PORTC** порт (**D0-D7**) младшего байт шины данных **LCD** панели.
- **PORTA** порт (**D8-D15**) старшего байт шины данных **LCD** панели.
- **PORTD** бит 4 (**CS**) передача сигнала доступа к **LCD** панели.
- **PORTD** бит 5 (**RD**) передача сигнала чтения из **LCD** панели.
- **PORTD** бит 6 (**WR**) передача сигнала записи в **LCD** панель.
- **PORTD** бит 7 (**RS**) передача сигнала команда/данные **LCD** панели.
- **PORTG** бит 0 (**RST**) передача сигнала сброса **LCD** панели.
- **PORTG** бит 1 (**BL**) передача сигнала подсветки **LCD** панели.



Панели LCD

На данный момент написаны драйвера для пяти **LCD** панелей:

1. **ILI9325** разрешение **320 x 240**
2. **ILI9341** разрешение **320 x 240**
3. **ILI9481** разрешение **480 x 320**
4. **ILI9486** разрешение **480 x 320**
5. **OTM8009A** разрешение **800 x 480**

Расположение выводов разъемов плат и панелей прилагаются в папке **“FSMC LCD”** в виде документов в формате **.xlsx**. Вполне возможно, что у Вас могут отличаться разъемы **LCD** панелей, но разъемы плат должны соответствовать.

Уникальный функционал и авторские алгоритмы

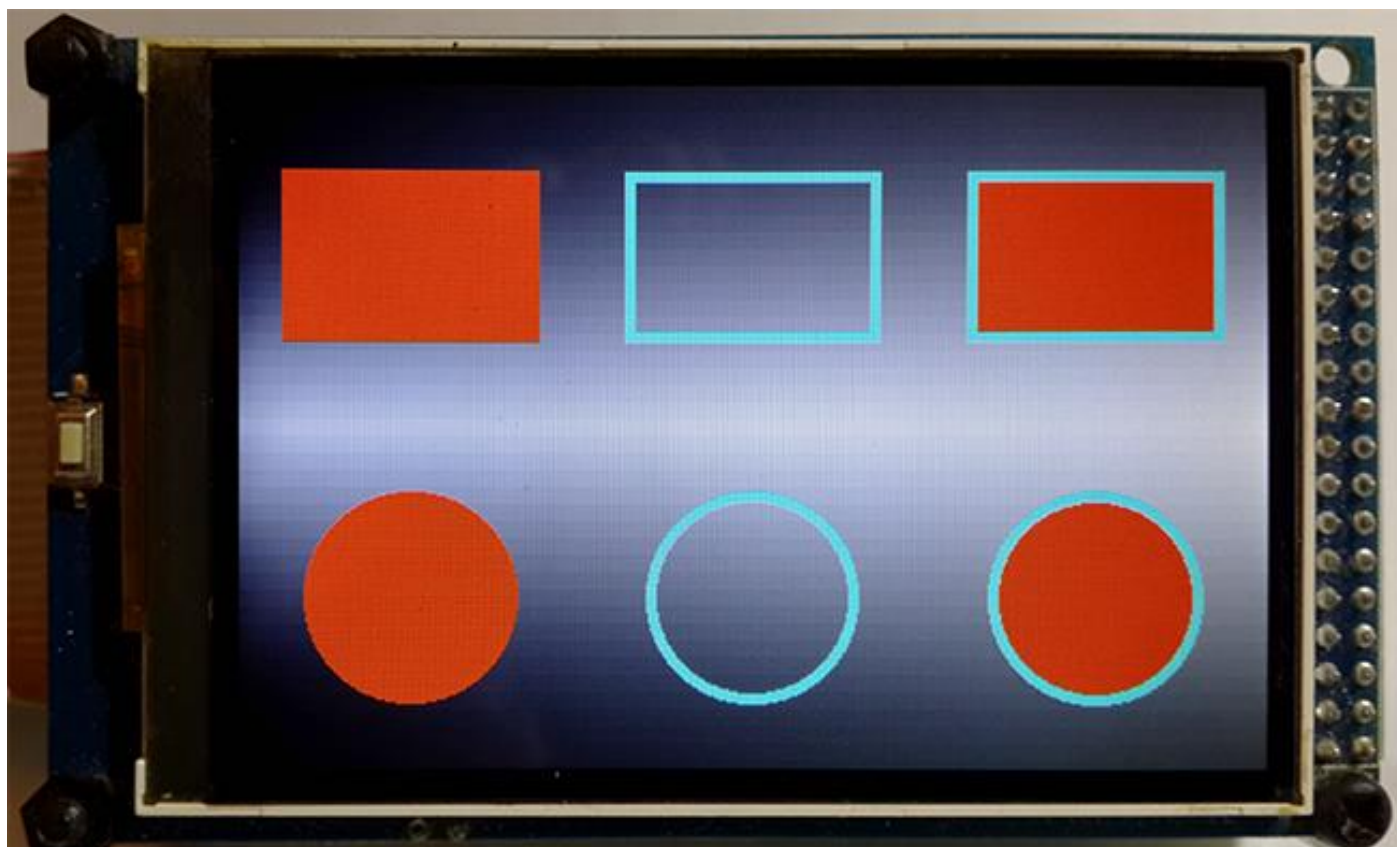


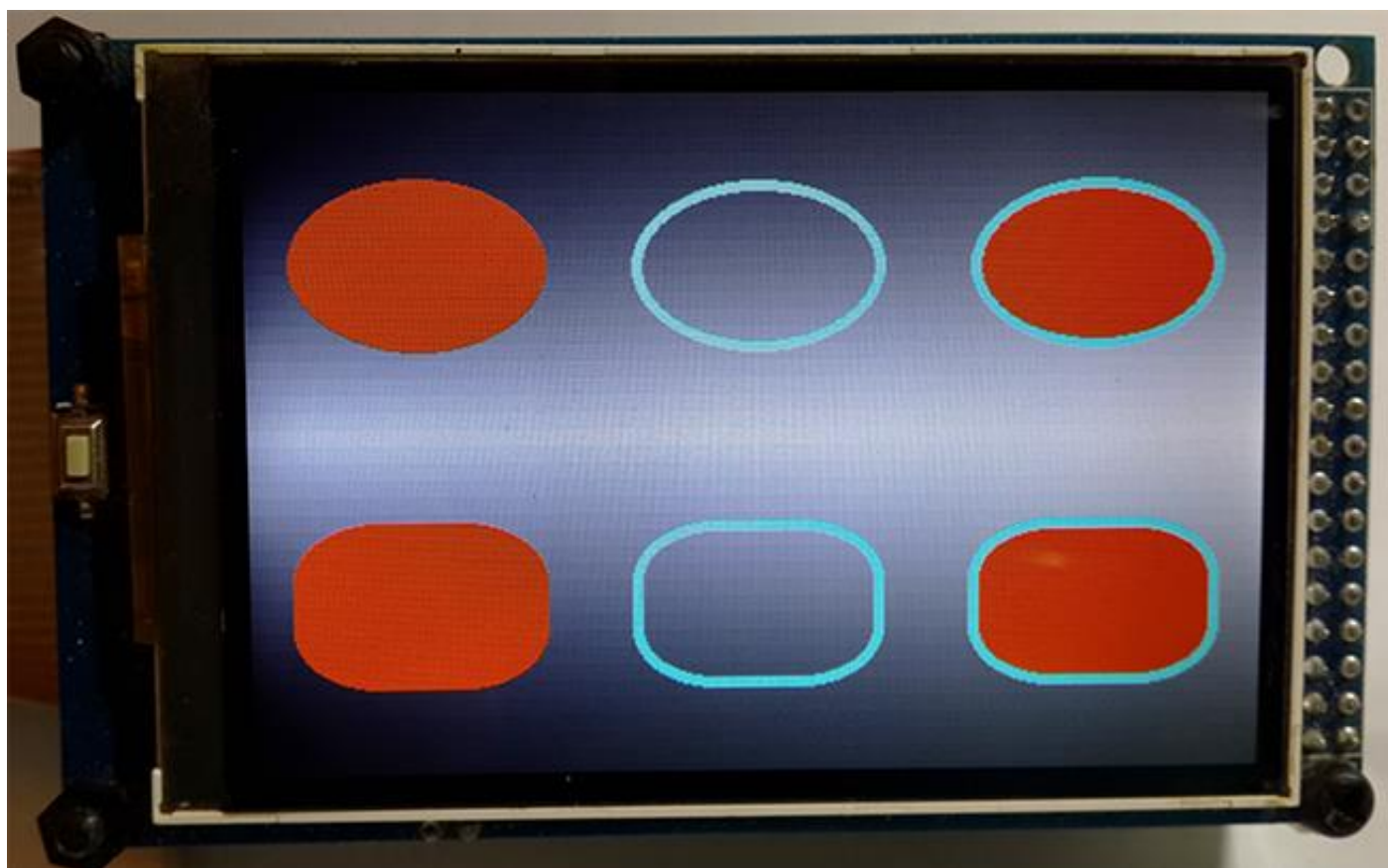
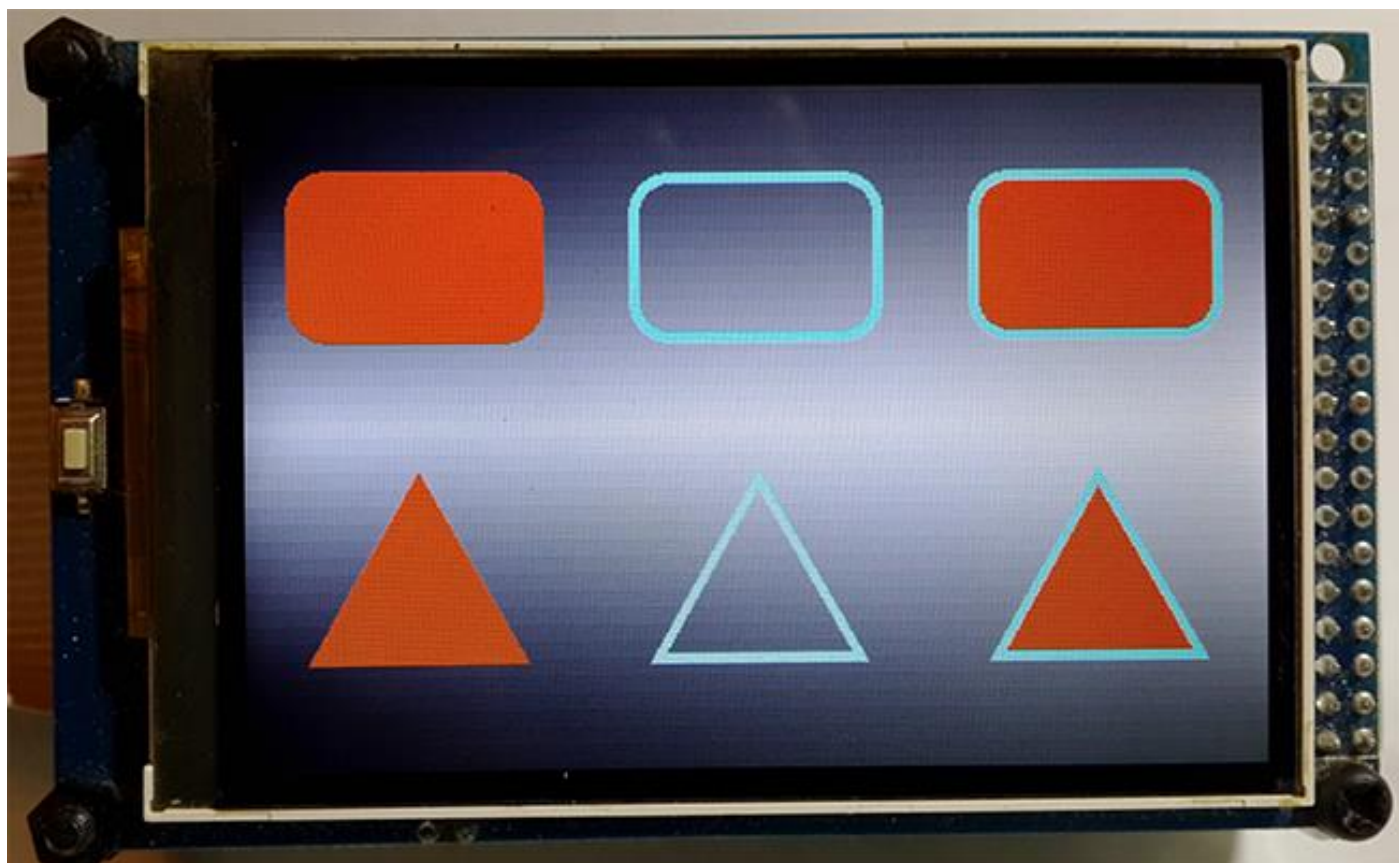
Основной задачей при создании библиотеки ставилось, кроме кроссплатформенности, максимально быстрая отрисовка графических объектов. В отличие от классических библиотек, использующих стандартный по пиксельный вывод, в данном проекте

реализованы собственные оптимизированные алгоритмы, разработанные с учетом архитектуры графической памяти (**GRAM**) дисплеев:

- **Скоростная отрисовка линий.** Используется авторский алгоритм, который рассчитывает сплошные прямые отрезки в наклонных линиях. Это позволяет минимизировать количество команд переключения адресов **GRAM LCD** панели и осуществлять быструю блочную заливку, что кратно увеличивает скорость отрисовки.
- **Отрисовка градиентным цветом.** Используется авторский алгоритм, который рассчитывает и заполняет градиентное окрашивание в вертикальном и горизонтальном направлении с линейной или центральной ориентацией за счет аппаратного ускорения **GRAM** матрицей **LCD** панели.
- **Многофункциональный трех поверхностный объект.** Используется авторский алгоритм, который рассчитывает в целочисленной математике окружность с заполнением градиентным цветом **на лету** не используя **фрейм-буферы**. В данном объекте заполняются и отрисовываются одновременно три округлые / круглые или прямоугольные поверхности градиентным цветом в разных направлениях **на лету**. Все три поверхности могут перерисовываться независимо друг от друга, это позволяет, акцентировать выбор объекта, перерисовывая другим цветом только одну, на пример, наружную поверхность не тратя время на отрисовку остальных поверхностей.

Продвинутые графические примитивы





Даже базовые геометрические фигуры в **KVB_LCD** сложнее стандартных аналогов. Они имеют:

- Регулируемый по ширине бордюр.
 - Центральную поверхность.
 - **Три режима отображения:** только контур, только внутренняя часть или объект целиком. Это позволяет выбирать на экране **LCD** панели объект, изменяя цвет бордюра, не тратя время на перерисовку внутренней поверхности. Прimitives типа треугольник (равносторонний и равнобедренный) имеют четыре ориентации вершины: 1 = левая, 2 = правая, 3 = вверх, 4 = вниз.
-

Продвинутая подсистема вывода текста



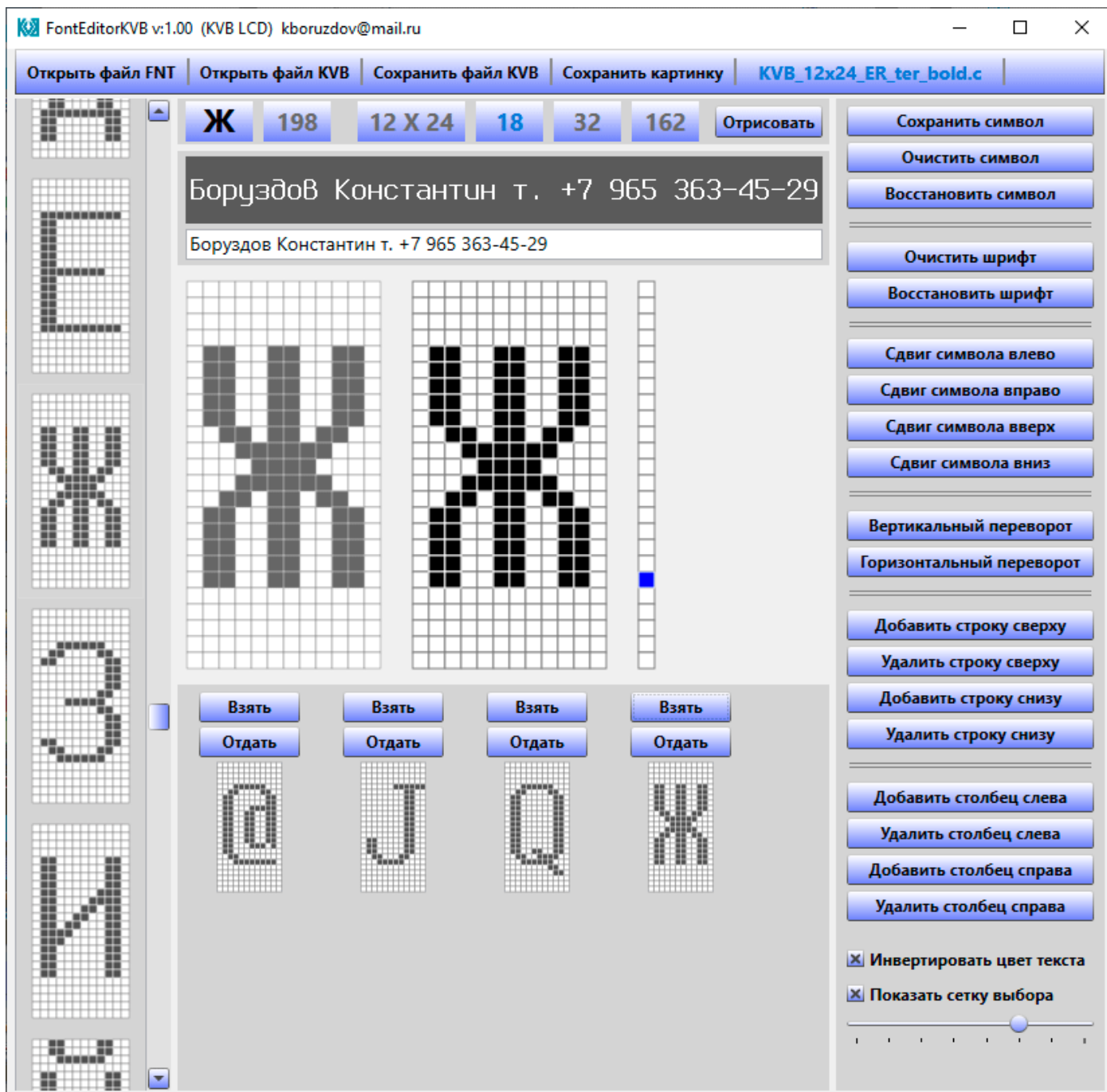


Движок обладает мощным и быстрым функционалом работы с текстом, который автоматически выравнивает строки и рендерит сложные визуальные эффекты **на лету** без использования **фрейм-буфера**:

- **Привязка к базовой линии.** Все шрифты привязываются к единой горизонтальной направляющей, нижняя полка символов. Это позволяет собирать одну строку из слов разного размера и цвета, гарантируя идеальную ровность текста.
- **Автоматический перенос длинного текста по словам.** Длинный непрерывный текст может состоять из множества текстовых, целочисленных и вещественных переменных, разного размера шрифта окрашенных разным цветом. Текст будет переноситься по словам на следующие строки если выходит за границу ширины текстового блока.
- **Поддержка табуляции.**
- **Установка кернинга шрифта** (меж символьное расстояние) с коррекцией длины пробелов и табуляции.
- **Расстояния между строками.** Расстояние задается как в плюс, так и в минус. Актуально в минус если длинный текст выводится только заглавными символами. Изменение расстояния в плюс приводит к лучшей читаемости обычного текста.
- **Коэффициент увеличения шрифта.**

- **Выравнивание текста.** Выравнивание задается по левому краю, правому краю или центру.
 - **Эффект тени.** Тень символа имеет четыре установки. 1 = тень низ справа, 2 = тень низ слева, 3 = тень верх справа, 4 = тень верх слева. Тем самым создается эффект выпуклости или вдавленности шрифта и лучшим образом влияет на читаемость текста.
 - **Круговой контур символов.** Четкий контур вокруг каждого символа для читаемости на любом фоне. Можно задать отображение только контура без символа.
 - **Градиентный текст.** Плавное окрашивание в горизонтальном или вертикальном направлении с помощью линейной или центральной заливки строки и применению тени или кругового контура.
-

Графический редактор шрифтов (Windows IDE)



Для создания и кастомизации экранных шрифтов разработано полноценное **Windows**-приложение, автоматизирующее весь цикл подготовки графических ресурсов:

- **Импорт DOS-шрифтов (.fnt).** На протяжении **DOS** эпохи было создано великое множество растровых шрифтов **.fnt** формата. Возможность быстрой загрузки классических растровых шрифтов формата **DOS** для их последующей адаптации под дисплеи микроконтроллеров.
- **Поточечное редактирование.** Удобный графический интерфейс для ручной отрисовки и корректировки каждого символа на пиксельной сетке с настройкой базовой линии.

- **Интерактивный рендеринг.** Отображение заданной текстовой строки **на лету** в реальном времени. Вы видите, как изменения конкретного пикселя в символе влияют на отображение целой строки.
- **Сохранение в Си-формат.** Генерация готовых `.c` файлов с ультра-компактной последовательной упаковкой данных (символ **9x16** занимает всего **18 байт**), полностью совместимых как с `PROGMEM` во Flash-памяти **ATmega128**, так и с обычными константными массивами в **STM32F407**.
- **Авторский пак.** С помощью данного редактора создано **23** уникальных русско-английских шрифта (**5** базовых включены в презентацию для демонстрации, полный набор и редактор доступны по запросу).

В демонстрационной версии редактора убрана возможность сохранения в **Си-формате**, полноценный редактор доступен по запросу.

С уважением,

Константин Боруздов.

Контактная информация:

 Email: kboruzdov@mail.ru

 Tel: +7 (965) 363-45-29
